



UNIVERSITÀ DI TRENTO

Department of Information Engineering and
Computer Science

Master's degree in Computer Science

DISTRIBUTED SYSTEMS
PROJECT 2025

MARS DISTRIBUTED STORAGE SYSTEM

Authors	Student ID
Dennis Cattoni	256139
Matteo Di Noia	258426

Academic year 2024/2025

Contents

1	Introduction	3
1.1	System condition	3
1.2	Our basic principle	3
2	System Architecture	3
2.1	Sub-states	4
2.2	State transitions	4
3	Membership states	4
3.1	Crash	4
3.2	Recover	4
3.3	Join	5
3.4	Leave	5
4	Normal and Sub states	7
4.1	Get	7
4.2	Update	7
5	Client consistency	7

1 Introduction

In this report, we explain the behavior and architecture choices of our distributed storage system.

1.1 System condition

We model the distributed storage system, under the assumptions provided by the professor, which define the system's operational guarantees and failure model. Specifically, we focus on, the fact that:

- At least N nodes are alive at any given time.
- At least one node in the system is not in the **CRASHED** state.
- The operations **Join**, **Leave**, **Crash** and **Recover** are atomic and execute in mutual exclusion with respect to other state transitions, and they happen only when no **Get/Update** is in progress.
- **Get** and **Update** operations can be invoked concurrently and may overlap in time.
- A client can communicate with only one node at a time and has no direct view of the system's internal state or topology.

The assumption that every operation (except **Get** and **Update**) is atomic means that the operation completely conclude reaching again a consistent state before any other operation starts.

Because of this, we can say that starting from an initially consistent state, and designing each operation to preserve this consistency, we can maintain always this as an invariant. Using the fact that this is an invariant, and that the operation are valid when this property holds; we can, then, show the correctness of the system under the professor assumption.

1.2 Our basic principle

In our system, we ensure that each node "holds" all possible keys it is currently responsible for, even if nobody ever wrote that key. That is possibly without having to actually store all possible key.

This property simplifies in particular recover operation, as it will be shown afterward. When a node A receive a request for a key value from another Node B (that is performing a read), it return a valid response with negative key. The way this is handled, is described in Chapter 4.1.

2 System Architecture

Our system is modeled as a finite state automaton, where there are the following states: **INITIAL**, **NORMAL**, **CRASHED**, **RECOVERING**, **JOINING**, **LEAVING**, and **LEFT** and two sub-states **SUB**, respectively **GET** and **UPDATE**. Each of these states represents a distinct phase in the node's life-cycle and defines the set of actions the node is allowed to perform.

2.1 Sub-states

The only particular one is **SUB**, which represent one of many sub-states of a single node in **NORMAL** can have. Each **SUB** is representing of a client operation (**Get/Update**) currently in progress.

The sub-states **Get** and **Update** are entered when the node, in **Normal** state, is actively serving read or write operations, ensuring that the main protocol logic is clearly separated from the logic of client interactions and transient behaviors.

2.2 State transitions

Transitions between states are triggered by control messages, internal timeouts or reaching some condition after receiving a message, and are designed to ensure that at every point the system maintains consistency. As shown in the following picture, the transitions between the existing states are:

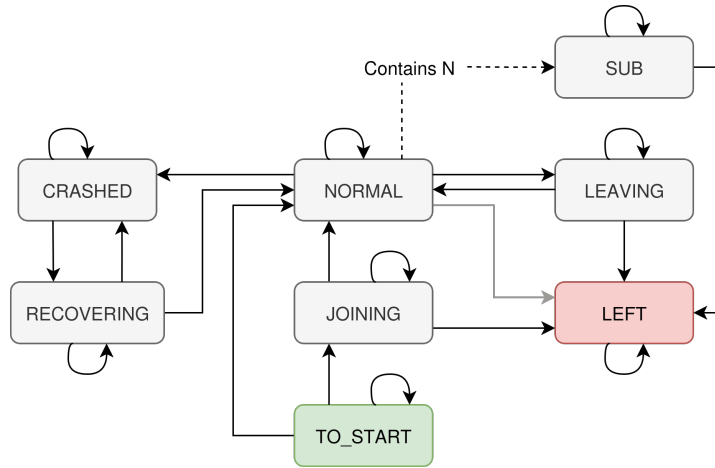


Figure 1: Finite state automaton of node state

This finite state design allows us to isolate and reason about the behavior of nodes in compartmentalized ways and allows us to only look at how a single state maintains consistency, so that is possible to demonstrate inductively that consistency is an invariant.

3 Membership states

3.1 Crash

When a **NORMAL** node receives a *Crash* message, it transitions to the **CRASHED** state and ignores all incoming messages except *Recover*. In addition, the transition to the **CRASHED** state is guaranteed to succeed and cannot fail.

3.2 Recover

When a **CRASHED** node receives *Recover* message, it changes to the **RECOVERING** state and asks its bootstrapping peer for the current ring members. If this last doesn't respond in

time, it will receive *Timeout* message and move back to the **CRASHED** state.

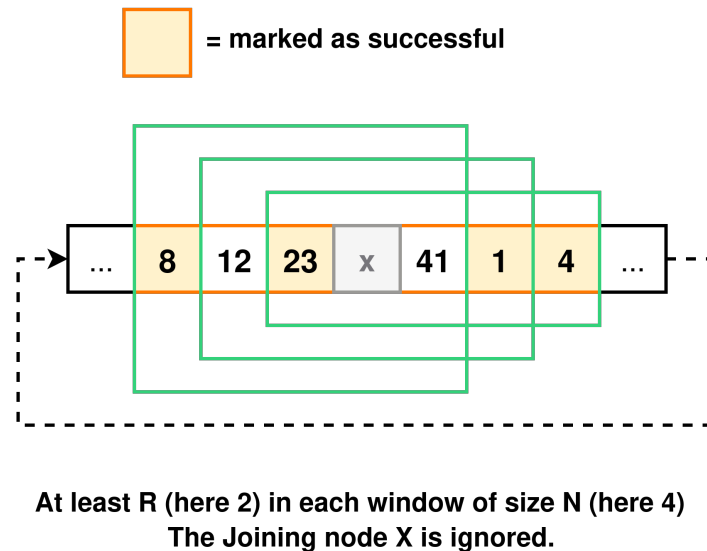
Instead, if it receives such a list, it drops the data entries that are no longer under its responsibility, because a node has joined while it was crashed.

On the other hand, we do not try to retrieve new data for keys that have become its responsibility due to a node leaving, as this is not necessary. The completion of a *Leave* operation guarantees that the quorum is still preserved. Moreover, based on our design principle, each node stores all the keys it is responsible for, possibly with a negative version, which indicates that the value has never been written on that node.

3.3 Join

When a **TO_START** node receives a *Join* message, the node transitions to the **JOINING** state and begins the bootstrap procedure in the same way as in the *Recover* process.

After this, it communicates with $N - 1$ nodes to its left and $N - 1$ nodes to its right in order to retrieve the data for which it is now responsible.



principles, means that W other nodes must contain the most recent write on such entry. In this case the W nodes must be in a set composed of $N - 1$ other responsible nodes and the new node that will be responsible once this leaves.

To do so, we perform a particular "write", called *RefreshIfNeeded* to these N nodes. The recipient must create or update the values passed if needed, and then Ack the message.

Eg: $R=2$ $W=3$ $N=4$

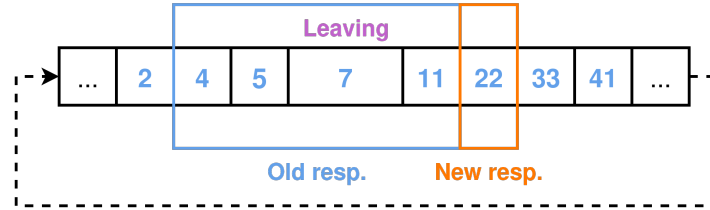


Figure 3: Example of members of leave operation

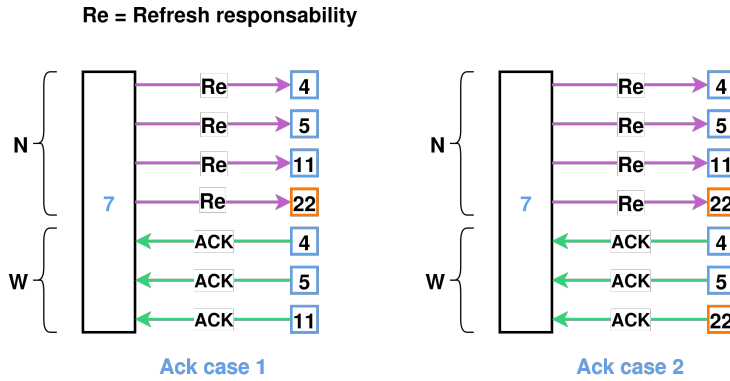


Figure 4: Two case of successful leave, with and without the new responsible member.

When W Ack are received for each key the leaving node contains, the node is free to leave. This holds because either W other nodes already contain the newer version, or the leaving node was one of the nodes that had the latest version, and after the leave, at least other W nodes have received and now store that newer version.

Note, that this need to be done for each key, but messages going to the same destination are merged to limits to the least amount of messages. Finally it is to note that this operation, could technically be done with less messages by sending only to the new responsible but that would reduce availability of such operation.

If a timeout is reached before completion, it informs all contacted nodes that it is no longer leaving through a *Rollback* message. These nodes then discard all data for which they are not responsible, as the leaving node has decided to remain. Therefore, they don't need to rollback also the version of the value as it has at most incremented and that cannot break the quorum if the state was consistent before. The node, then, switches back to the NORMAL state.

4 Normal and Sub states

4.1 Get

When a node in **NORMAL** receive a *Get* message, it creates a new **SUB** state that handle completely such operation.

The new sub-state communicates with the responsible nodes and when it receives R valid reply, it respond to the client. If the quorum obtained has a negative version, it means that the element in question does not exist, and it reports as such to the client.

Instead, in case of timeout, it just forward this information to the client.

4.2 Update

Similarly to the *Get*, when a node in **NORMAL** receive an *Update* message, it creates a new **SUB** state that handle completely such operation.

The **UPDATE** sub-state contains three phases:

1. It first tries to acquire write locks and retrieve the current version number. When W responses are received, this means the node has successfully acquired enough locks and can proceed to the write phase.
2. To prevent clients from reading partially written values, the node then requests read locks from the same W nodes. This step is performed only after ensuring that the write can proceed. This operation cannot fail, since by assumption no node can crash during this phase.
3. Finally, the node writes the new value to the W nodes and releases all locks.

Also note that the write and read lock are both per node and per key.

When releasing locks, we must be careful to always release all of them, even if some nodes did not respond (yet or because they are **CRASHED**) to the lock request. This ensures that no lock is left hanging, which could block future operations forever. And this work based on the professor assumption that during this phase no node can crash.

If a timeout occurs before the end of the first phase, the node explicitly releases all the locks it had attempted to acquire; again also including that didn't respond. On the other hand, if the timeout is triggered after the first phase has completed, it is simply ignored. The operation is now guaranteed to complete correctly and in a timely manner.

5 Client consistency

While Sequential Consistency is easily shown to be ensured through version numbers and the fact that a single write can complete at the time, maintaining client-side consistency requires the client to keep track of the last version it has observed. During a read or write operation, the client sends this version along with the request. If the version stored on the system is lower than the expected one, the operation is aborted and simply times out.